In [2]:
```python
import math
import numpy as np
from matplotlib import pyplot as plt
import copy
import gif
from IPython.display import Image
```

In [3]:
```python
def animate(data, j, patch, head, joints):
    first = []
    second = []
    for i in joints:
        first.append(data[j][0][i])
        second.append(data[j][1][i])
    a = np.array([first, second])
    a = np.transpose(a)
    patch.set_xy(a)
    head.center = (data[j][0][5], data[j][1][5])
    return patch, head

# To create a gif image
@gif.frame
def plot_samples_normal(data, j):
    # draw the simulation
    fig = plt.figure()
    fig.set_dpi(100)
    fig.set_size_inches(12, 3)

    ax = plt.axes(xlim=(-1, 10), ylim=(-0.1, 3))


    joints = [5, 0, 1, 2, 1, 0, 3, 4]
    patch = plt.Polygon([[0, 0], [0, 0]], closed=None, fill=None, edgecolor="k")
    head = plt.Circle((0, 0), radius=0.15, fc="k", ec="k")
    ax.add_patch(patch)
    ax.add_patch(head)
    plt.title(f"total_distance: {data[j][0][5]:.2f}")
    animate(data, j, patch, head, joints)
    # ax.Line2D([-0.5, 0], [10, 0], color='black')

    step_size = 11 / len(data)

    # Add a progress bar rectangle at the bottom
    progress_bar_height = 0.1
    progress_bar_y = -0.1  # Position of the progress bar on the y-axis
    progress_bar = plt.Rectangle((-1, progress_bar_y), step_size*j, progress_bar_height, color='blue')
    ax.add_patch(progress_bar)

    # Add a border for the progress bar
    progress_bar_border = plt.Rectangle((-1, progress_bar_y), 11, progress_bar_height, fill=False, edgecolor='black', linewidth=1)
    ax.add_patch(progress_bar_border)
```

In [4]:
```python
# plan is an array of 40 floating point numbers
def sim(plan):
    """
    Simulates the motion of a multi-jointed body based on a given plan.

    Parameters:
    plan (list of float): A list of control parameters for the simulation. Each element should be between -1 and 1.

    Returns:
    tuple:
        - data (list of list of list of float): A list of positions of the body parts at each time step. Each position is a list of two list
        - final_position (float): The final x-coordinate of the head (body part 5).

    The simulation involves:
    - Adjusting the control parameters to be within the range [-1, 1].
    - Initializing physical properties such as mass, edge lengths, spring constants, and damping factors.
    - Iteratively updating the positions and velocities of the body parts based on forces, torques, and constraints.
    - Handling contact with the ground and applying friction.
    - Recording the positions of the body parts at each time step.
    - Returning the recorded data and the final x-coordinate of the head.
    """
    for i in range(0, len(plan)):
        if plan[i] > 1:
            plan[i] = 1.0
        elif plan[i] < -1:
            plan[i] = -1.0

    dt = 0.1 # time step
    friction = 1.0 # friction coefficient
    gravity = 0.1 # gravity constant
    mass = [30, 10, 5, 10, 5, 10] # mass of body parts
    edgel = [0.5, 0.5, 0.5, 0.5, 0.9] # edge lengths
    edgesp = [160.0, 180.0, 160.0, 180.0, 160.0]    # spring constants
    edgef = [8.0, 8.0, 8.0, 8.0, 8.0]   # damping factors
    anglessp = [20.0, 20.0, 10.0, 10.0] # angular spring constants
    anglesf = [8.0, 8.0, 4.0, 4.0]  # angular damping factors

    edge = [(0, 1), (1, 2), (0, 3), (3, 4), (0, 5)] # edges
    angles = [(4, 0), (4, 2), (0, 1), (2, 3)] # angles
```

```python
    # vel and pos of the body parts, 0 is hip, 5 is head, others are joints
    v = [[0.0, 0.0, 0.0, 0.0, 0.0, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]]
    p = [[0, 0, -0.25, 0.25, 0.25, 0.15], [1, 0.5, 0, 0.5, 0, 1.9]]

    spin = 0.0
    maxspin = 0.0
    lastang = 0.0

    data = []

    for j in range(20): # 20 time steps
        for k in range(10): # 10 substeps
            lamb = 0.05 + 0.1 * k
            t0 = 0.5
            if j > 0:
                t0 = plan[2 * j - 2]
            t0 *= 1 - lamb
            t0 += plan[2 * j] * lamb

            t1 = 0.0
            if j > 0:
                t1 = plan[2 * j - 1]
            t1 *= 1 - lamb
            t1 += plan[2 * j + 1] * lamb

            contact = [False, False, False, False, False, False]
            for z in range(6):
                if p[1][z] <= 0:
                    contact[z] = True
                    spin = 0
                    p[1][z] = 0

            anglesl = [-(2.8 + t0), -(2.8 - t0), -(1 - t1) * 0.9, -(1 + t1) * 0.9]

            disp = [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
            dist = [0, 0, 0, 0, 0]
            dispn = [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
            for z in range(5):
                disp[0][z] = p[0][edge[z][1]] - p[0][edge[z][0]]
                disp[1][z] = p[1][edge[z][1]] - p[1][edge[z][0]]
                dist[z] = (
                    math.sqrt(disp[0][z] * disp[0][z] + disp[1][z] * disp[1][z]) + 0.01
                )
                inv = 1.0 / dist[z]
                dispn[0][z] = disp[0][z] * inv
                dispn[1][z] = disp[1][z] * inv

            dispv = [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
            distv = [0, 0, 0, 0, 0]
            for z in range(5):
                dispv[0][z] = v[0][edge[z][1]] - v[0][edge[z][0]]
                dispv[1][z] = v[1][edge[z][1]] - v[1][edge[z][0]]
                distv[z] = 2 * (disp[0][z] * dispv[0][z] + disp[1][z] * dispv[1][z])

            forceedge = [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
            for z in range(5):
                c = (edgel[z] - dist[z]) * edgesp[z] - distv[z] * edgef[z]
                forceedge[0][z] = c * dispn[0][z]
                forceedge[1][z] = c * dispn[1][z]

            edgeang = [0, 0, 0, 0, 0]
            edgeangv = [0, 0, 0, 0, 0]
            for z in range(5):
                edgeang[z] = math.atan2(disp[1][z], disp[0][z])
                edgeangv[z] = (dispv[0][z] * disp[1][z] - dispv[1][z] * disp[0][z]) / (
                    dist[z] * dist[z]
                )

            inc = edgeang[4] - lastang
            if inc < -math.pi:
                inc += 2.0 * math.pi
            elif inc > math.pi:
                inc -= 2.0 * math.pi
            spin += inc
            spinc = spin - 0.005 * (k + 10 * j)
            if spinc > maxspin:
                maxspin = spinc
                lastang = edgeang[4]

            angv = [0, 0, 0, 0]
            for z in range(4):
                angv[z] = edgeangv[angles[z][1]] - edgeangv[angles[z][0]]

            angf = [0, 0, 0, 0]
            for z in range(4):
                ang = edgeang[angles[z][1]] - edgeang[angles[z][0]] - anglesl[z]
                if ang > math.pi:
                    ang -= 2 * math.pi
                elif ang < -math.pi:
                    ang += 2 * math.pi
                m0 = dist[angles[z][0]] / edgel[angles[z][0]]
                m1 = dist[angles[z][1]] / edgel[angles[z][1]]
                angf[z] = ang * anglessp[z] - angv[z] * anglesf[z] * min(m0, m1)
```

```
        edgetorque = [[0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
        for z in range(5):
            inv = 1.0 / (dist[z] * dist[z])
            edgetorque[0][z] = -disp[1][z] * inv
            edgetorque[1][z] = disp[0][z] * inv

        for z in range(4):
            i0 = angles[z][0]
            i1 = angles[z][1]
            forceedge[0][i0] += angf[z] * edgetorque[0][i0]
            forceedge[1][i0] += angf[z] * edgetorque[1][i0]
            forceedge[0][i1] -= angf[z] * edgetorque[0][i1]
            forceedge[1][i1] -= angf[z] * edgetorque[1][i1]

        f = [[0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0]]
        for z in range(5):
            i0 = edge[z][0]
            i1 = edge[z][1]
            f[0][i0] -= forceedge[0][z]
            f[1][i0] -= forceedge[1][z]
            f[0][i1] += forceedge[0][z]
            f[1][i1] += forceedge[1][z]

        for z in range(6):
            f[1][z] -= gravity * mass[z]
            invm = 1.0 / mass[z]
            v[0][z] += f[0][z] * dt * invm
            v[1][z] += f[1][z] * dt * invm

            if contact[z]:
                fric = 0.0
                if v[1][z] < 0.0:
                    fric = -v[1][z]
                    v[1][z] = 0.0

                s = np.sign(v[0][z])
                if v[0][z] * s < fric * friction:
                    v[0][z] = 0
                else:
                    v[0][z] -= fric * friction * s
            p[0][z] += v[0][z] * dt
            p[1][z] += v[1][z] * dt

        data.append(copy.deepcopy(p))

        if contact[0] or contact[5]:
            return data, p[0][5]
    return data, p[0][5]
```

In [8]:

```
import numpy as np

def simulated_annealing(plan, iterations=5000, initial_temp=100, cooling_rate=0.99):
    """
    Simulated Annealing to optimize QWOP plan.

    Parameters:
    - plan: Initial plan (list of 40 floats between -1 and 1).
    - iterations: Number of iterations for optimization.
    - initial_temp: Initial temperature for annealing.
    - cooling_rate: Rate at which the temperature cools.

    Returns:
    - best_plan: Optimized plan achieving the maximum distance.
    - best_distance: The maximum distance achieved by the best plan.
    """
    current_plan = plan[:]
    current_distance = sim(current_plan)[1]  # Evaluate initial distance
    best_plan = current_plan[:]
    best_distance = current_distance

    temperature = initial_temp

    for i in range(iterations):
        # Generate a new candidate plan
        new_plan = current_plan[:]
        idx = np.random.randint(len(plan))
        new_plan[idx] += np.random.uniform(-0.1, 0.1)  # Slight perturbation
        new_plan[idx] = max(-1.0, min(1.0, new_plan[idx]))  # Keep within bounds

        # Evaluate the new plan
        new_distance = sim(new_plan)[1]

        # Decide whether to accept the new plan
        delta_distance = new_distance - current_distance
        if delta_distance > 0 or np.random.uniform(0, 1) < np.exp(delta_distance / temperature):
            current_plan = new_plan
            current_distance = new_distance

            # Update the best plan if the new plan is better
            if new_distance > best_distance:
                best_plan = new_plan
                best_distance = new_distance
```

```
        # Cool down the temperature
        temperature *= cooling_rate

    return best_plan, best_distance


# Initialize a random plan
initial_plan = [np.random.uniform(-1, 1) for _ in range(40)]
```

In [9]:
```
# Run Simulated Annealing
optimized_plan_sa, optimized_distance_sa = simulated_annealing(
    plan=initial_plan, iterations=5000, initial_temp=500, cooling_rate=0.99
)

# Simulate the best plan to generate the gif
data, total_distance = sim(optimized_plan_sa)
print("Simulated Annealing Total Distance = ", total_distance)

# Create and save the gif
frames = []
for j in range(len(data)):
    frame = plot_samples_normal(data, j)
    frames.append(frame)

gif.save(frames, "sa_animation.gif", duration=40)
Image(filename="sa_animation.gif")
```

```
Simulated Annealing Total Distance =  4.882405398498132
```

Out[9]:
```
<IPython.core.display.Image object>
```

In [7]:
```
def evolutionary_algorithm(
    population_size=50, generations=100, mutation_rate=0.1, elite_fraction=0.2
):
    """
    Evolutionary Algorithm to optimize QWOP plan.

    Parameters:
    - population_size: Number of individuals in the population.
    - generations: Number of generations to evolve.
    - mutation_rate: Probability of mutating each plan entry.
    - elite_fraction: Fraction of top individuals retained in each generation.

    Returns:
    - best_plan: Optimized plan achieving the maximum distance.
    - best_distance: The maximum distance achieved by the best plan.
    """
    # Initialize a random population of plans
    population = [
        [np.random.uniform(-1, 1) for _ in range(40)] for _ in range(population_size)
    ]

    # Evaluate initial population
    def evaluate_population(pop):
        return [sim(individual)[1] for individual in pop]

    def crossover(parent1, parent2):
        # Single-point crossover
        crossover_point = np.random.randint(1, len(parent1) - 1)
        return parent1[:crossover_point] + parent2[crossover_point:]

    def mutate(individual):
        # Mutate individual with a small chance
        for i in range(len(individual)):
            if np.random.uniform(0, 1) < mutation_rate:
                individual[i] += np.random.uniform(-0.1, 0.1)
                individual[i] = max(-1.0, min(1.0, individual[i]))  # Keep within bounds
        return individual

    for generation in range(generations):
        # Evaluate population
        scores = evaluate_population(population)

        # Rank individuals by score
        ranked_population = sorted(zip(scores, population), key=lambda x: x[0], reverse=True)
        elite_count = int(elite_fraction * population_size)

        # Select elites
        new_population = [ind for _, ind in ranked_population[:elite_count]]

        # Generate offspring from elites
        while len(new_population) < population_size:
            parent1, parent2 = np.random.choice(elite_count, 2, replace=False)
            offspring = mutate(crossover(ranked_population[parent1][1], ranked_population[parent2][1]))
            new_population.append(offspring)

        # Replace population with new generation
        population = new_population

    # Return the best individual from the final generation
    final_scores = evaluate_population(population)
```

```
    best_index = np.argmax(final_scores)
    return population[best_index], final_scores[best_index]
```

In [6]:
```
# Run Evolutionary Algorithm
optimized_plan_ea, optimized_distance_ea = evolutionary_algorithm(
    population_size=500, generations=500, mutation_rate=0.2, elite_fraction=0.25
)

# Simulate the best plan to generate the gif
data, total_distance = sim(optimized_plan_ea)
print("Evolutionary Algorithm Total Distance = ", total_distance)

# Create and save the gif
frames = []
for j in range(len(data)):
    frame = plot_samples_normal(data, j)
    frames.append(frame)

gif.save(frames, "ea_animation.gif", duration=40)
Image(filename="ea_animation.gif")
```

```
Evolutionary Algorithm Total Distance =  9.519580226433801
```

Out[6]:
```
<IPython.core.display.Image object>
```

In [ ]: